

# Test

Sono stati realizzati i test unitari per tutte le classi implementate. I test sono stati realizzati cercando di garantire una copertura ottimale del codice. Quando ritenuto opportuno sono stati utilizzati dei test double. Infine sono stati realizzati dei test di integrazione ponendo l'attenzione sui flussi relativi a ciascun caso d'uso. Di seguito un resoconto dei test effettuati.

## TriggerStato

- TC01: `testCreazioneValida` : Verifica che un json ben formattato venga letto e correttamente iniettato negli attributi interni della classe. Trigger creato correttamente.
- TC02: `testCreazioneJsonMalformato` : Verifica il comportamento in caso di json malformato. Se l'utente o la UI inviano un JSON incompleto o malformato, il sistema respinge la creazione sollevando un'eccezione, evitando comportamenti anomali successivi.
- TC03: `testCreazioneParametriNull` : Verifica il comportamento del costruttore nel caso vengano passati dei parametri `null` . Una eccezione deve essere lanciata.
- TC04: `testIsSoddisfattoTargetSbagliato` : Testa il comportamento del metodo `isSoddisfatto()` nell'eventualità di un `idTarget` diverso da quello del target associato al trigger. In tale circostanza anche qualora la condizione del trigger risulti soddisfatta l'evento deve essere ignorato. Il metodo deve restituire `false` .
- TC05: `testIsSoddisfattoParametroSbagliato` : Testa il comportamento del metodo `isSoddisfatto()` nell'eventualità di un `paramOsservato` diverso da quello specificato nel trigger. L'evento deve essere ignorato. Il metodo deve restituire `false` .
- TC06: `testIsSoddisfattoCondizioneVera` : Simula l'arrivo di un evento valido con `idTarget` e `paramOsservato` che combaciano con quelli specificati nel trigger e che soddisfi la condizione di attivazione di quest'ultimo. Verifica che il metodo restituisca `true` .
- TC07: `testIsSoddisfattoCondizioneFalsa` : Simula l'arrivo di un evento valido con `idTarget` e `paramOsservato` che combaciano con quelli specificati nel trigger ma che non soddisfa la condizione di attivazione. Verifica che il metodo restituisca `false` .
- TC08: `testIsSoddisfattoOperatoreNonValido` : Verifica il comportamento nel caso di operatore non valido. Il metodo deve restituire `false` .

## TriggerTemporale

- TC01: `testCreazioneValidaConGiorni` : Controlla che l'orario e la lista dei giorni vengano estratti dal json e convertiti correttamente nei tipi nativi di Java ( `LocalTime` e `DayOfWeek` ).
- TC02: `testCreazioneOrarioSbagliato` : Controlla il comportamento nell'eventualità di un'orario non valido. Un' eccezione deve essere lanciata.
- TC03: `testCreazioneGiornoSbagliato` : Controlla il comportamento nell'eventualità di un giorno non valido. Un' eccezione deve essere lanciata.
- TC04: `testGetDelay_Future_noDays` :  
**Nessun giorno di ripetizione specificato e orario trigger nel futuro rispetto all'orario attuale:** la routine dovrà attivarsi oggi all'orario specificato. Il delay deve essere calcolato di conseguenza.
- TC05: `testGetDelay_Past_noDays` :  
**Nessun giorno di ripetizione specificato e orario trigger nel passato rispetto all'orario attuale:** la routine dovrà attivarsi domani all'orario specificato. Il delay deve essere calcolato di conseguenza.
- TC06: `testGetDelay_Future_withDays_todayInList` :  
**Lista giorni ripetizione specificata, oggi è in lista e l'orario specificato è nel futuro:** la routine dovrà attivarsi oggi all'orario specificato. Il delay deve essere calcolato di conseguenza.
- TC07: `testGetDelay_Past_withDays_todayInList` :  
**Lista giorni ripetizione specificata, oggi è in lista e l'orario specificato è nel passato:** la routine dovrà attivarsi il prossimo giorno specificato nella lista all'orario specificato. Il delay deve essere calcolato di conseguenza.
- TC08: `testGetDelay_withDays_todayNotList` :  
**Lista giorni ripetizione specificata, oggi non è in lista:** la routine dovrà attivarsi il prossimo giorno presente in lista a partire da oggi all'orario specificato. Il delay deve essere calcolato di conseguenza.

## Routine

- TC01: `testCostruttore` : Verifica che la routine sia istanziata correttamente ipotizzando che i parametri siano validi. Inoltre verifica che ogni routine, al momento della sua nascita, sia abilitata.
- TC02: `testCostruttoreSenzaNome` : Verifica il comportamento del costruttore qualora il parametro `nome` fosse `null` o non valido. Una eccezione deve essere sollevata.
- TC03: `testCostruttoreNullParams` : Verifica il comportamento del costruttore qualora qualcuno dei parametri fosse `null` . Una eccezione deve essere sollevata.

## MotoreRoutine

- TC01: `testAddRoutine` : Verifica l'aggiunta di una routine alla lista tenuta dal MotoreRoutines.
- TC02: `testOnEvento_HappyPath` : Verifica del comportamento della classe alla ricezione di un evento valido supponendo la routine attivata e la condizione soddisfatta. Il comando associato alla routine deve essere eseguito dal `ControllerTargets` .
- TC03: `testOnEvento_CondizioneNonSoddisfatta` : Verifica del comportamento della classe alla ricezione di un evento valido supponendo la routine attivata ma la condizione non soddisfatta. Il comando associato alla routine non deve essere eseguito dal `ControllerTargets` .
- TC04: `testOnEvento_RoutineDisabilitata` : Verifica del comportamento della classe alla ricezione di un evento valido supponendo la routine disabilitata. Il comando associato alla routine non deve essere eseguito dal `ControllerTargets` .
- TC05: `testSchedule_RoutineTemporaleAbilitata` : Verifica l'attivazione di una routine temporale abilitata. Il comando associato alla routine deve essere eseguito dal `ControllerTargets` .
- TC06: `testSchedule_RoutineTemporaleDisabilitata` : Verifica l'attivazione di una routine temporale disabilitata. Il comando associato alla routine non deve essere eseguito dal `ControllerTargets` .

## ControllerRoutines

- TC01: `testCostruttore_nullParams` : Verifica che i parametri del costruttore non siano `null` . Lancia un'eccezione.
- TC02: `testAddRoutine_Time_HappyPath` : Verifica l'aggiunta di una routine temporale. La routine deve essere registrata correttamente nel `MotoreRoutine` .
- TC03: `testAddRoutine_Event_HappyPath` : Verifica l'aggiunta di una routine di stato. La routine deve essere registrata correttamente nel `MotoreRoutine` .
- TC04: `testAddRoutine_TargetNonTrovato` : Verifica l'aggiunta di una routine specificando un `idTarget` inesistente. La routine non deve essere registrata e deve essere lanciata un'eccezione.
- TC05: `testAddRoutine_TargetOsservatoNonTrovato` : Verifica l'aggiunta di una routine specificando un `idTargetOsservato` inesistente. La routine non deve essere registrata e deve essere lanciata un'eccezione.
- TC06: `testAddRoutine_TipoSconosciuto` : Verifica l'aggiunta di una routine specificando un tipo di routine inesistente. La routine non deve essere registrata e deve essere lanciata un'eccezione.
- TC07: `testAddRoutine_ComandoIncompatibile` : Verifica l'aggiunta di una routine specificando un comando incompatibile. La routine non deve essere registrata e deve essere lanciata un'eccezione.

## ControllerMonitoraggio

- TC01: `testMsgRetePing` : Verifica comportamento all'arrivo di un messaggio di `PING` valido. Se il dispositivo era `OFFLINE` deve tornare `ONLINE` .
- TC02: `testMsgReteCambioStato` : Verifica comportamento all'arrivo di un messaggio di `CAMBIO_STATO` valido. Il dispositivo deve cambiare il suo stato in accordo con il `payload` del `msgRete` .
- TC03: `testWatchdog` : Verifica meccanismo rilevazione inattività. Se viene rilevato un dispositivo inattivo da troppo tempo deve essere considerato `OFFLINE` .
- TC04: `testMsgRete_TargetSconosciuto` : Ricezione messaggio con target sconosciuto. Deve essere ignorato.
- TC05: `testMsgRete_TipoMessaggioSconosciuto` : Ricezione messaggio con tipo sconosciuto. Deve essere ignorato.
- TC06: `testAddMonitorListener_IgnoraNull` : Aggiunta di `MonitorListener` `null`. Un'eccezione deve essere lanciata.

Inoltre, a seguito di modifiche, alcuni test cases sono stati modificati o aggiunti alle seguenti classi.

## ServizioReteTCP

- TC-02: `testSend_RispostaError` : Verifica comportamento metodo `send()` a seguito di ricezione risposta d'errore. Un'eccezione deve essere lanciata.
- TC-04: `testSend_RispostaOK` : Verifica comportamento metodo `send()` a seguito di ricezione risposta `OK` . Tutto ok non deve succedere nulla.
- TC-05: `testListen_RicezioneMessaggio` : Verifica la ricezione di un messaggio da parte di un dispositivo fisico. Il messaggio deve essere ricevuto correttamente.
- TC-06: `testListen_JsonMalformato` : Verifica lo scenario di ricezione di un messaggio contenente un json malformato da parte di un dispositivo fisico. Un'eccezione viene lanciata ma il server continua a funzionare.
- TC-07: `testListen_CampiMancanti` : Verifica lo scenario di ricezione di un messaggio con campi mancanti da parte di un dispositivo fisico. I campi mancanti vengono segnati come `UNKNOWN` (questi msg verranno scartati dal metodo `msgRete`) ma il server continua a funzionare.
- TC08: `testSend_PortaNonNumerica` : Verifica comportamento metodo `send()` quando viene specificata una porta non numerica. Un'eccezione deve essere lanciata.

## Comando

- TC-03: `testCostruttoreJson_HappyPath` : Verifica costruzione tramite json del comando. Istanza costruita correttamente.
- TC-04: `ctestCostruttoreJson_StringaVuota` : Verifica costruzione tramite json del comando. Un'eccezione deve essere lanciata.
- TC-05: `testCostruttoreJson_JsonMalformato` : Verifica costruzione tramite json del comando. Un'eccezione deve essere lanciata.

## RegistroTargets

- TC08: `testGetDispositivi_EstraeSenzaDuplicati` : Verifica del metodo `getDispositivi()`. Tutti i dispositivi (non i gruppi) presenti nel `RegistroTargets` devono essere restituiti senza duplicati.

## ControllerTargets

- TC01: `testEseguiComando_TargetNonTrovato` : Test esecuzione comando su target non registrato nel sistema. Un'eccezione deve essere lanciata: il comando non deve partire.
- TC02: `testEseguiComando_NessunDispositivoCompatibile` : Test esecuzione comando incompatibile ai target registrati nel sistema. Il comando non deve partire.
- TC03: `testEseguiComando_Successo` : Test esecuzione comando valido. Il messaggio contenente il comando deve essere inviato al dispositivo fisico.
- TC04: `testEseguiComando_ErroreDiComunicazione` : Test esecuzione comando valido. Il comando dovrebbe essere inviato ma si verifica un errore di rete. Viene sollevata un eccezione.

## Test Integrazione

Sono stati realizzati dei test di integrazione per ciascun caso d'uso. I test di integrazione sono raccolti nella classe `SystemIntegrationTest.java` e comprendono tutti i casi d'uso finora implementati:

- UC02: Comanda Dispositivi
- UC06a: Crea Routine
- UC06b: Attiva Routine
- UC07: Monitoraggio: `msgRete()`
- UC07 Monitoraggio: `verificaInattivita()`